Python Quick Guide (part 1) / Python-pikaopas (osa 1)

Vesa Kaarnioja
vesa.kaarnioja@lut.fi

LUT School of Engineering Sciences / LUT insinööritieteiden tiedekunta

September 8, 2026 / 8. syyskuuta 2026

Practical Matters

- In addition to mathematical topics, this course includes a programming component using the Python programming language for scientific computing.
- Python learning materials can be found under the "Python course notes" tab on the course webpage.
- The Python learning materials are intended for self-study, but we will also have three brief lectures during this course covering the main points during Weeks 2, 4, and 6.

Käytännön asioita

- Matemaattisten aiheiden lisäksi tällä kurssilla harjoitellaan myös Python-ohjelmointikielen käyttämistä tieteellisessä laskennassa.
- Python-oppimateriaalit löytyvät kurssin verkkosivun "Python-moniste"-välilehdeltä.
- Python-oppimateriaalit on tarkoitettu itseopiskeluun, mutta kurssin aikana järjestetään myös kolme lyhyttä luentoa aiheesta viikoilla 2, 4 ja 6.

Python Exercises

- Starting this week, there will be weekly Python groups covering Python exercises. You can find the first Python exercise sheet under the Week 2 tab on the course page.
- Each Python exercise sheet contains 4 tasks. It is recommended to try to complete at least the first task before the exercise session. The remaining tasks can be solved during the session.
- **Points for completing Python exercises can only be earned by participating in a Python exercise session.**
- You are required to complete at least 25% of this course's Python exercises. Completing more Python exercises beyond this quota earns you points toward your grade.

Python-harjoitukset

- Tästä viikosta lähtien järjestetään viikoittaisia Python-ryhmiä, joissa käydään läpi Python-harjoitustehtäviä. Ensimmäinen Python-tehtäväpaperi löytyy kurssisivun Viikko 2 -välilehdeltä.
- Jokaisessa Python-tehtäväpaperissa on 4 tehtävää. Tehtävä 1 kannattaa pyrkiä ratkaisemaan jo ennen ryhmään osallistumista. Muut tehtävät voi hyvin jättää ratkaistavaksi harjoitusryhmässä.
- **Python-tehtävistä voi saada pisteitä vain Python-ryhmään osallistumalla.**
- Vähintään 25 % tämän kurssin Python-harjoituksista on suoritettava. Tämän kiintiön ylittävien harjoitusten ratkaisemisesta saa lisäpisteitä kurssin arvosteluun.

Python Programming Language

- Python is a general-purpose programming language. It is designed to be easy to read and write.
- Free, open-source, used everywhere in science, engineering, and data analysis.
- Python's functionality can be extended by a wide range of libraries.

NumPy	**SciPy**	**Matplotlib**
Arrays, linear algebra, numerical routines	Integration, differential equations, optimization, statistics	Plots and visualizations

These three libraries cover everything you need in this course.

Python-ohjelmointikieli

- Python on yleiskäyttöinen ohjelmointikieli. Se on suunniteltu helppolukuiseksi ja helposti kirjoitettavaksi.
- Ilmainen, avoimen lähdekoodin kieli, jota käytetään laajalti tieteessä, tekniikassa ja data-analyysissä.
- Pythonin toiminnallisuutta voidaan laajentaa monipuolisilla kirjastoilla.

NumPy	**SciPy**	**Matplotlib**
Taulukot, lineaarialgebra, numeeriset rutiinit	Integrointi, differentiaaliyhtälöt, optimointi, tilastotiede	Kuvaajat ja visualisoinnit

Nämä kolme kirjastoa kattavat kaiken tarvittavan tällä kurssilla.

Installation

Recommended set-up:

1. **Download Python** from `python.org`
   Latest stable **3.x** version, tick "Add Python to PATH" during install.
2. **Download VS Code** from `code.visualstudio.com`
   Install the Python extension (by Microsoft) from the Extensions panel.
3. **Install the libraries:** open a terminal and run:

```
pip install numpy scipy matplotlib
```

(or `pip3 install numpy scipy matplotlib`)

If you have trouble with the installation, you can ask for help during this week's tutorial groups:

- Tue 8.9., 10-12, Student Union building room 216 (immediately after today's lecture)
- Wed 9.9., 13-14, Student Union building room 216 (tomorrow)

Asennus

Suositeltu asennustapa:

1. **Lataa Python** osoitteesta `python.org`
   Uusin vakaa **3.x**-versio, valitse asennuksen aikana "Add Python to PATH".
2. **Lataa VS Code** osoitteesta `code.visualstudio.com`
   Asenna Python-laajennus (Microsoftilta) Laajennukset-paneelista.
3. **Asenna kirjastot:** avaa terminaali ja suorita:

   ```
   pip install numpy scipy matplotlib
   ```

(tai `pip3 install numpy scipy matplotlib`)

Jos sinulla on ongelmia asennuksen kanssa, voit pyytää apua tämän viikon harjoitusryhmissä:

- Ti 8.9., 10–12, Yo-talo huone 216 (heti tämän päivän luennon jälkeen)
- Ke 9.9., 13–14, Yo-talo huone 216 (huomenna)

Running Python Code

Three ways to run Python:

Interactive terminal	**Script file (.py)**	**VS Code**
Type `python`. Commands execute line by line. Great for quick calculations.	Write in a `.py` file, run with `python myfile.py`. Reproducible.	Open a `.py` file, press **Run Python File** (or **F5**). Output appears in the terminal.

First script:

```
print("Hello World!")
```

Python-koodin ajaminen

Kolme tapaa ajaa Python-koodia:

Interaktiivinen terminaali

Kirjoita `python`. Komennot suoritetaan rivi kerrallaan. Sopii nopeisiin laskuihin.

Skriptitiedosto (.py)

Kirjoita `.py`-tiedostoon, aja komennolla `python myfile.py`. Toistettava.

VS Code

Avaa `.py`-tiedosto, paina **Run Python File** (tai **F5**). Tuloste näkyy terminaalissa.

Ensimmäinen skripti:

```
print("Hello World!")
```

Python as a Calculator

Standard mathematical operations in Python:

```
2 + 3        # addition           = 5
10 - 4       # subtraction        = 6
3 * 7        # multiplication     = 21
22 / 7       # division           = 3.14285...
2 ** 10      # exponentiation     = 1024
```

Note

Use ** for powers, not ^

Python laskimena

Tavalliset laskutoimitukset Pythonissa:

```
2 + 3          # yhteenlasku          = 5
10 - 4         # vähennyslasku        = 6
3 * 7          # kertolasku           = 21
22 / 7         # jakolasku            = 3.14285...
2 ** 10        # potenssiinkorotus    = 1024
```

Huomio

Käytä ** potenssiin, ei ^

Assignment, Equality, and Floating-point Arithmetic

Assignment vs. equality

In Python = assigns a value while == tests equality.

```
x = 5          # assigns 5 to x
x == 5         # True  (tests whether x equals 5)
x == 6         # False
```

Never compare floats with ==

Floating-point arithmetic is not exact, so == can give surprising results:

```
0.1 + 0.2 == 0.3          # False!
print(0.1 + 0.2)          # 0.30000000000000004
abs(0.1 + 0.2 - 0.3) < 1e-10   # True (correct way)
```

Sijoitus, yhtäsuuruus ja liukulukuaritmetiikka

Sijoitus vs. yhtäsuuruus

Pythonissa = sijoittaa arvon muuttujaan, kun taas == testaa yhtäsuuruutta.

```
x = 5          # sijoittaa arvon 5 muuttujaan x
x == 5         # True  (testaa, onko x yhtä kuin 5)
x == 6         # False
```

Älä koskaan vertaile liukulukuja ==:lla

Liukulukuaritmetiikka ei ole tarkkaa, joten == voi antaa yllättäviä tuloksia:

```
0.1 + 0.2 == 0.3          # False!
print(0.1 + 0.2)          # 0.30000000000000004
abs(0.1 + 0.2 - 0.3) < 1e-10   # True (oikea tapa)
```

Mathematical Functions with NumPy

Prefer `numpy` over the built-in `math`: it works on both scalars and arrays.

```
import numpy as np

np.exp(1)            # e^1 ≈ 2.71828
np.log(np.e)         # natural log  = 1
np.log10(1000)       # log base 10  = 3
np.log2(8)           # log base 2   = 3
np.sqrt(2)           # sqrt ≈ 1.41421

np.sin(np.pi / 6)    # sin(pi/6) = 0.5
np.cos(np.pi)        # cos(pi)   = -1
np.tan(np.pi / 4)    # tan(pi/4) = 1
```

Note

Angles are always in radians. Use `np.pi` for π.

Matemaattiset funktiot NumPyllä

Kannattaa suosia `numpyä` sisäänrakennetun `mathin` sijaan: se toimii sekä skalaarien että taulukoiden kanssa.

```
import numpy as np

np.exp(1)           # e^1 ≈ 2.71828
np.log(np.e)        # luonnollinen logaritmi = 1
np.log10(1000)      # log kantaluku 10 = 3
np.log2(8)          # log kantaluku 2  = 3
np.sqrt(2)          # neliöjuuri ≈ 1.41421

np.sin(np.pi / 6)   # sin(pi/6) = 0.5
np.cos(np.pi)       # cos(pi)   = -1
np.tan(np.pi / 4)   # tan(pi/4) = 1
```

Huomio

Kulmat ovat aina radiaaneissa. Käytä `np.pi` arvoa π varten.

Defining Functions: def

Numerical functions can be defined just as in mathematics.

```
import numpy as np

def f(x):
    return x**2 + 2*x + 1

def g(x, y):
    return np.exp(-x**2) * np.cos(y)

print(f(3))       # 16
print(g(0, 0))    # 1.0
print(f(10))      # 121
```

Note on the syntax of def

`def name(arguments):` → note the colon!

Body is indented (4 spaces). `return` gives back the result.

Funktioiden määrittely: def

Numeerisia funktioita voidaan määritellä aivan kuten matematiikassakin.

```
import numpy as np

def f(x):
    return x**2 + 2*x + 1

def g(x, y):
    return np.exp(-x**2) * np.cos(y)

print(f(3))       # 16
print(g(0, 0))    # 1.0
print(f(10))      # 121
```

Huomio def-syntaksista

`def nimi(argumentit):` → huomaa kaksoispiste!

Runko on sisennetty (4 välilyöntiä). `return` palauttaa tuloksen.

One-Line Functions: `lambda`

Simple expressions can be written alternatively as a `lambda` function:

```
import numpy as np

f = lambda x: x**2 + 2*x + 1

g = lambda x, y: np.exp(-x**2) * np.cos(y)

print(f(3))       # 16
print(g(0, 0))    # 1.0
print(f(10))      # 121
```

Use `lambda` for short throwaway functions and `def` for anything you reuse.

Yhden rivin funktiot: `lambda`

Yksinkertaiset lausekkeet voidaan määritellä `lambda-funktion` avulla:

```
import numpy as np

f = lambda x: x**2 + 2*x + 1

g = lambda x, y: np.exp(-x**2) * np.cos(y)

print(f(3))       # 16
print(g(0, 0))    # 1.0
print(f(10))      # 121
```

Käytä `lambdaa` lyhyisiin kertakäyttöfunktioihin ja `defiä` kaikkeen, mitä käytät uudelleen.

First Plot: $f(x) = x^2$

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3, 3, 200)  # 200 points in [-3, 3]
y = x**2

plt.plot(x, y)
plt.xlabel('x'); plt.ylabel('y')
plt.title('A simple parabola')
plt.grid()
plt.show()
```

`np.linspace(a,b,n)`: creates n equally spaced values in $[a, b]$. Operations like `x**2` act on every element at once.

Ensimmäinen kuvaaja: $f(x) = x^2$

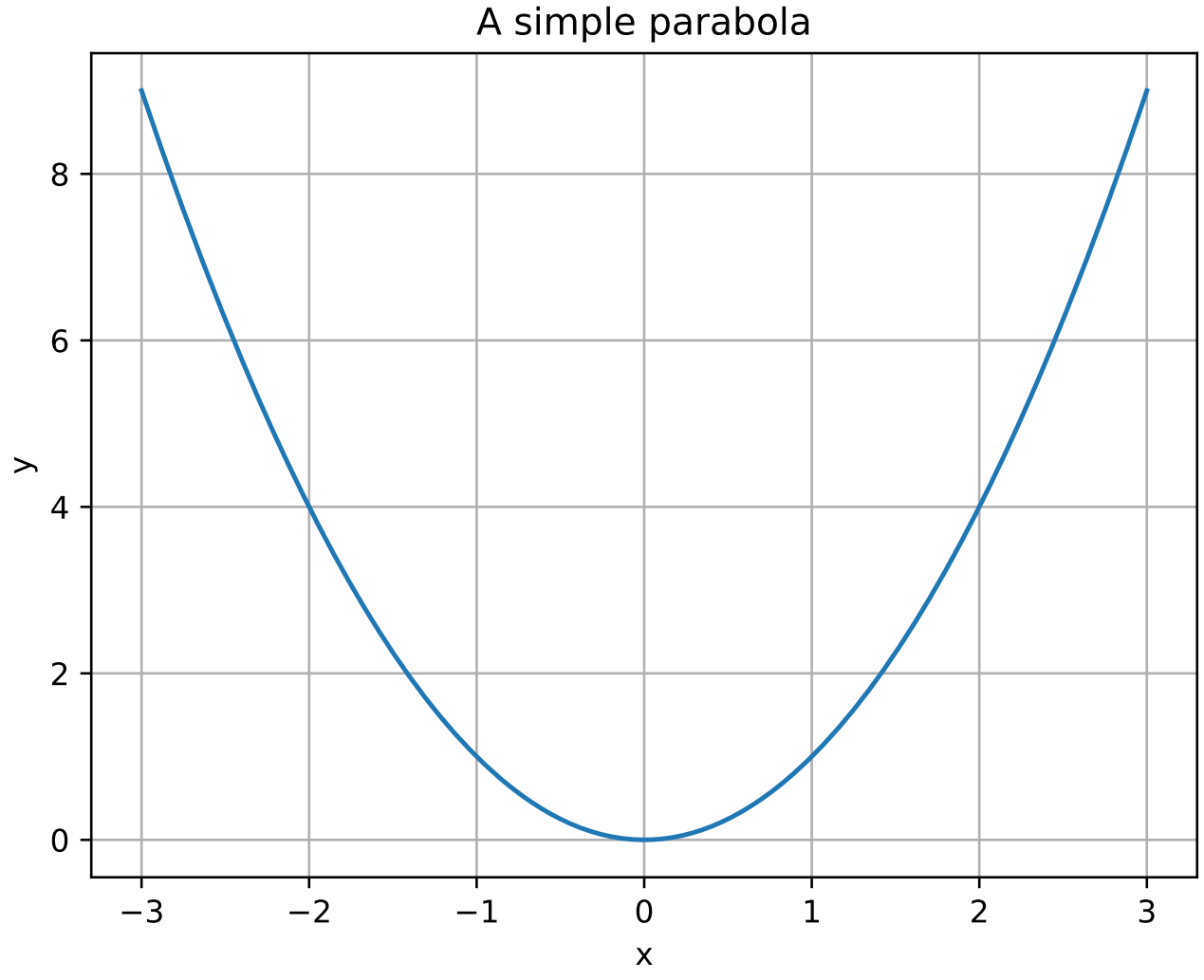
```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3, 3, 200)  # 200 pistettä välillä [-3, 3]
y = x**2

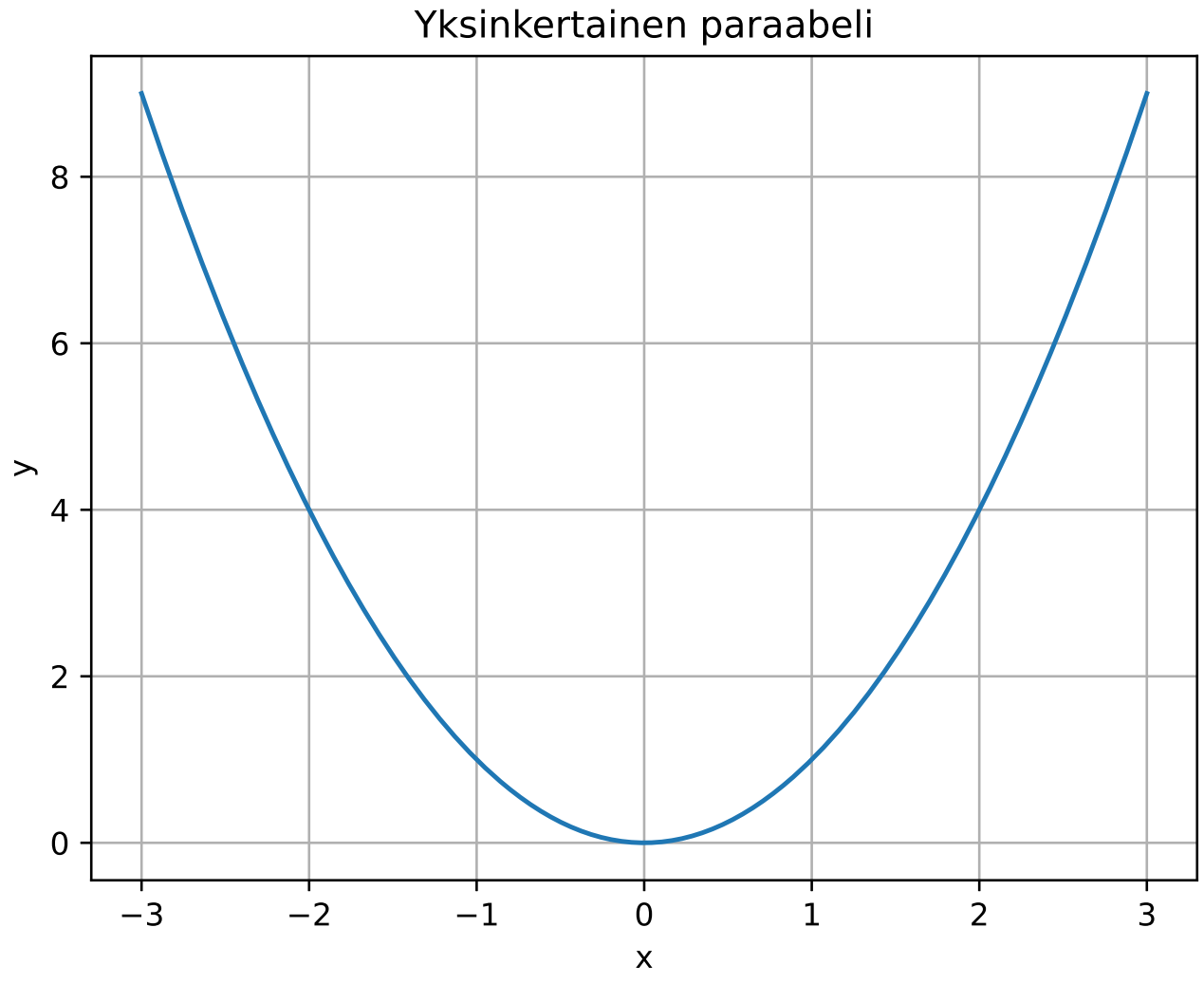
plt.plot(x, y)
plt.xlabel('x'); plt.ylabel('y')
plt.title('Yksinkertainen paraabeli')
plt.grid()
plt.show()
```

`np.linspace(a,b,n)`: laatii n tasavälistä pistettä välillä $[a, b]$. Operaatiot kuten `x**2` kohdistuvat jokaiseen vektorin `x` alkioon.

Figure Created by the Previous Script



Edellisen skriptin muodostama kuvaaja



NumPy Arrays

A NumPy array is a mathematical vector supporting element-wise operations.

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
b = np.array([0., 1., 2., 3., 4.])

print(a + b)       # [1. 3. 5. 7. 9.]
print(a * 2)       # [ 2  4  6  8 10]
print(a**2)        # [ 1  4  9 16 25]
print(np.sum(a))   # 15

k = np.arange(1, 6)         # [1, 2, 3, 4, 5]
x = np.linspace(0, 1, 5)    # [0. 0.25 0.5 0.75 1.]
```

Note

Observe that `np.arange(1,6)` gives `[1,2,3,4,5]`. In Python, ranges are typically interpreted as semiopen intervals $[a, b)$, where the start value a is included but the stop value b is not.

NumPy-taulukot

NumPy-taulukko on matemaattinen vektori, joka tukee alkiokohtaisia operaatioita.

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
b = np.array([0., 1., 2., 3., 4.])

print(a + b)       # [1. 3. 5. 7. 9.]
print(a * 2)       # [ 2  4  6  8 10]
print(a**2)        # [ 1  4  9 16 25]
print(np.sum(a))   # 15

k = np.arange(1, 6)         # [1, 2, 3, 4, 5]
x = np.linspace(0, 1, 5)    # [0. 0.25 0.5 0.75 1.]
```

Huomio

Huomaa, että `np.arange(1,6)` tuottaa vektorin `[1,2,3,4,5]`. Pythonissa välit tulkitaan tyypillisesti puoliavoimina väleinä $[a, b)$, joissa alkuarvo a sisältyy väliin mutta loppuarvo b ei.

Computing Sums

Recall the Taylor series for e^x:

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$$

For $x = 1$ truncated at $k = 10$:

$$e \approx \sum_{k=0}^{10} \frac{1}{k!} = 1 + 1 + \tfrac{1}{2} + \tfrac{1}{6} + \tfrac{1}{24} + \cdots + \tfrac{1}{3628800}$$

Strategy

1. Build an array of terms $1/k!$ for $k = 0, 1, \ldots, N$.
2. Sum with `np.sum()`.

Summien laskeminen

Palautetaan mieleen eksponenttifunktion Taylor-sarja:

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$$

Arvolle $x = 1$ katkaistuna kohdassa $k = 10$:

$$e \approx \sum_{k=0}^{10} \frac{1}{k!} = 1 + 1 + \tfrac{1}{2} + \tfrac{1}{6} + \tfrac{1}{24} + \cdots + \tfrac{1}{3628800}$$

Strategia

1. Muodosta vektori termeistä $1/k!$ arvoilla $k = 0, 1, \ldots, N$.
2. Laske termit yhteen `np.sum()`-funktiolla.

Computing Sums

```
import numpy as np

# Approach 1: compute factorials by hand
factorials = np.array([1, 1, 2, 6, 24, 120,
                       720, 5040, 40320, 362880, 3628800])
terms = 1.0 / factorials
print("Sum:", np.sum(terms))          # ≈ 2.7182818...
print("True e:", np.e)

# Approach 2: use SciPy to compute factorials
from scipy.special import factorial
k = np.arange(0, 11)                  # k = 0,...,10
terms = 1.0 / factorial(k)
print("Sum:", np.sum(terms))
```

The partial sum converges to $e \approx 2.71828\ldots$ very rapidly.

Summien laskeminen

```
import numpy as np

# Tapa 1: lasketaan kertomat käsin
factorials = np.array([1, 1, 2, 6, 24, 120,
                       720, 5040, 40320, 362880, 3628800])
terms = 1.0 / factorials
print("Summa:", np.sum(terms))      # ≈ 2.7182818...
print("Todellinen e:", np.e)

# Tapa 2: lasketaan kertomat SciPyllä
from scipy.special import factorial
k = np.arange(0, 11)                # k = 0,...,10
terms = 1.0 / factorial(k)
print("Summa:", np.sum(terms))
```

Osasumma suppenee arvoon $e \approx 2.71828\ldots$ hyvin nopeasti.